# Assignment 2 — Part 2

BUC and Attribute-Oriented Induction

September 14, 2026

## 1 Overview

Part 2 implements and evaluates **two fundamental data-cube algorithms** on the census dataset (`M26_DA_A2_Part2.csv`, 299,285 tuples):

1. **BUC (Bottom-Up Cubing)** — in-memory and **out-of-core (paged)** variants, computing the **iceberg cube** (minsup-pruned) with the classic optimizations (dimension reordering, pre-count pruning / Apriori property, constant-dimension shortcut, node collapsing).
2. **AOI (Attribute-Oriented Induction)** — attribute removal, concept-tree climbing, and threshold control, producing **characteristic rules** and **class-comparison (discriminant) rules** with **t-weights and d-weights**.

**Table 1:** Part 2 deliverables and their locations.

| **Deliverable** | **Location** |
|---|---|
| BUC in-memory + optimizations | `part2/src/buc.py` |
| BUC out-of-core (paged, disk spill) | `part2/src/buc_paged.py` |
| AOI + class comparison | `part2/src/aoi.py` |
| Concept hierarchies (10 attributes) | `part2/hierarchies/*.csv` |
| Experiment driver | `part2/src/run_experiments.py` |
| Correctness vs DuckDB CUBE | `part2/outputs/correctness_inmem.csv`, `correctness_paged.csv` |
| Benchmarks (minsup / dims / memory / optimizations) | `part2/outputs/benchmark_*.csv` |
| AOI rules + comparison | `part2/outputs/aoi_rules.csv`, `aoi_char_rules_population.csv`, `aoi_thresholds.csv`, `buc_vs_aoi.csv` |
| Plots (7 PNGs) | `part2/outputs/plots/*.png` |

Reproduce everything with: `python part2/src/run_experiments.py` (add `--quick` for a fast smoke run).

## 2 Dataset and Encoding

- **299,285 tuples** × 12 categorical dimensions (`age_group`, `class_of_worker`, `education`, `marital_status`, `race`, `sex`, `employment_status`, `tax_filer_status`, `household_summary`, `citizenship`, `hispanic_origin`, `weeks_worked_band`) plus the class label `income` (>50K / ≤50K).
- All dimensions are **integer-encoded** (`numpy` codes) so BUC partitions and sorts on compact arrays; the measure is `COUNT(*)` (fully additive).
- Default iceberg threshold: **minsup** = 50.

# 3 BUC — Bottom-Up Cubing

## 3.1 Algorithm

BUC computes the iceberg cube bottom-up: it recursively partitions the data on one dimension at a time, and for each partition recurses on the remaining dimensions. Because a cell can only meet minsup if all its ancestors do (**Apriori property**), BUC prunes entire subtrees early.

Our implementation (`part2/src/buc.py`) includes:

1. **Dimension reordering** — dimensions sorted by **descending cardinality** (most selective first), so partitions shrink fastest and pruning triggers earliest.
2. **Pre-count pruning** — before recursing into a value's partition, its size is compared against minsup; small partitions are skipped without sorting.
3. **Constant-dimension shortcut** — when a partition is constant on a remaining dimension, that dimension is emitted as `ALL` without further recursion (10,759 skips in the 8-dim run).
4. **Node collapsing** — sibling cells with identical counts are collapsed into one output row (19 collapsed nodes).

## 3.2 Out-of-core (paged) BUC

`part2/src/buc_paged.py` implements BUC under a **fixed memory budget**:

- Input is stored as fixed-width binary **pages** (8,192 rows/page) on disk.
- A partition whose estimated size exceeds the budget is processed by **external merge-sort** (chunked sort + spill to temp files + k-way merge), then recursion continues on the sorted runs.
- Statistics track pages read, bytes read/written, temp files, and the number of disk vs. memory children.

**Verification:** the paged variant produces **exactly the same cells and counts** as the in-memory variant at every minsup tested (see `benchmark_minsup.csv`: identical `inmem_cells == paged_cells` for all 8 settings, e.g. 86,043 cells at minsup = 50).

## 3.3 Correctness vs DuckDB `GROUP BY CUBE`

Every non-empty combination of the first 8 dimensions (255 combinations) was computed by both BUC variants and compared cell-by-cell against DuckDB's `GROUP BY CUBE ... HAVING count >= minsup`:

- **In-memory BUC: 255 / 255 combinations match** (identical cell sets and counts).
- **Paged BUC: 13 / 13 spot-checked combinations match.**

(`part2/outputs/correctness_inmem.csv`, `correctness_paged.csv`)

### 3.4 Benchmarks

**Runtime vs minsup** (8 dimensions, 299,285 tuples; `benchmark_minsup.csv`):

**Table 2:** BUC runtime and cube size vs. iceberg threshold minsup.

| minsup | in-mem (s) | cells | paged (s) |
|---|---|---|---|
| 5 | 22.96 | 296,463 | 22.62 |
| 10 | 13.38 | 208,286 | 22.19 |
| 25 | 9.67 | 127,772 | 19.62 |
| 50 | 8.81 | 86,043 | 17.41 |
| 100 | 7.75 | 55,664 | 16.45 |
| 250 | 7.19 | 29,700 | 14.50 |
| 500 | 6.57 | 17,841 | 13.62 |
| 1000 | 5.70 | 10,123 | 12.87 |

→ Higher minsup prunes more subtrees: the cube shrinks **29**× (296K → 10K cells) and in-mem runtime drops **4**× (23.0 s → 5.7 s). The paged variant pays a constant I/O overhead (≈2× slower) but scales below any memory budget.

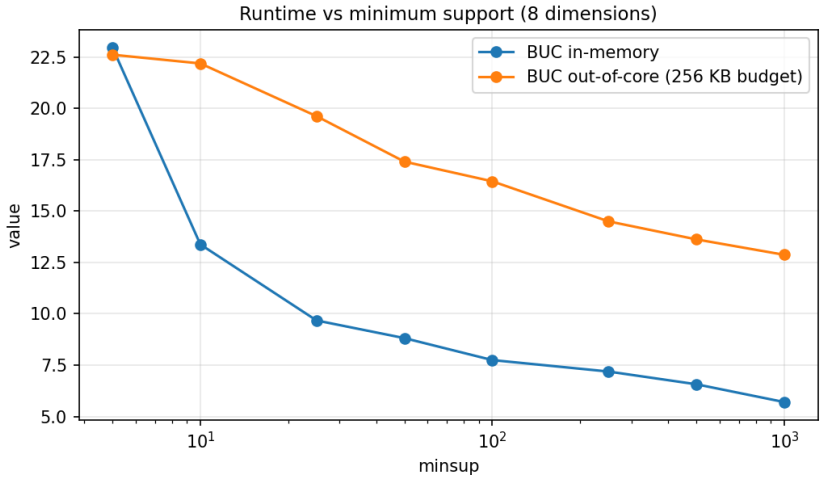



**Figure 1:** BUC runtime vs. iceberg threshold (minsup).

**Runtime vs number of dimensions** (`benchmark_dims.csv`, minsup = 50):

**Table 3:** BUC cost vs. number of cube dimensions (minsup = 50).

| dims | runtime (s) | cells | recursive calls |
|---|---|---|---|
| 2 | 0.10 | 72 | 72 |
| 4 | 0.51 | 2,177 | 2,177 |
| 6 | 1.87 | 13,548 | 13,548 |
| 8 | 8.43 | 86,043 | 86,043 |
| 10 | 47.74 | 432,722 | 432,722 |

→ Runtime grows roughly **exponentially** with dimension count (as expected: $2^d$ grouping sets), ≈6× per added pair of dimensions.

**Paged BUC vs memory budget** (`benchmark_memory.csv`, 8 dims, minsup = 50):

**Table 4:** Paged (out-of-core) BUC under varying memory budgets.

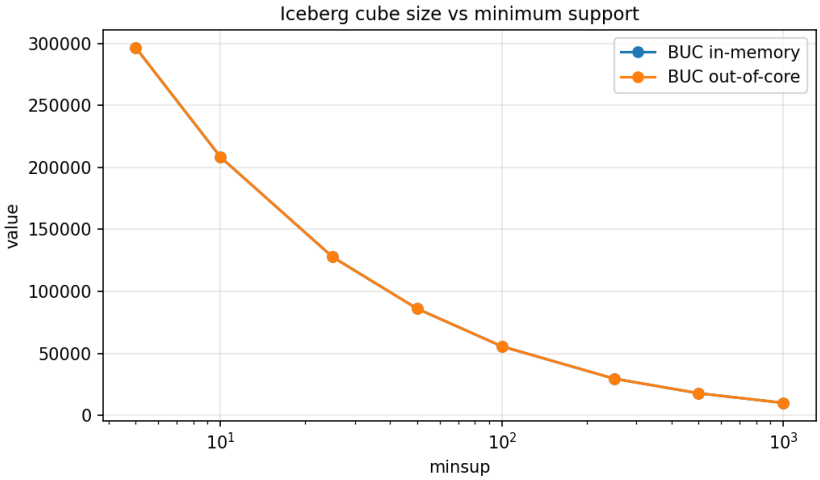



**Figure 2:** Number of iceberg-cube cells vs. minsup.

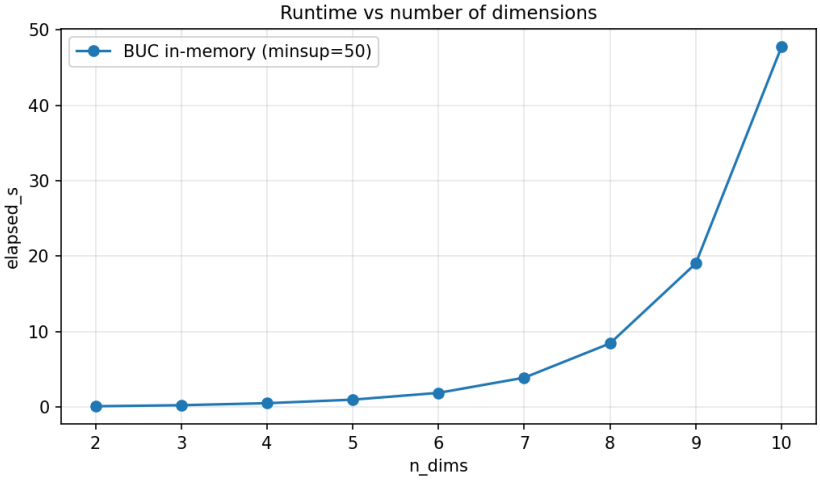



**Figure 3:** BUC runtime vs. number of dimensions.

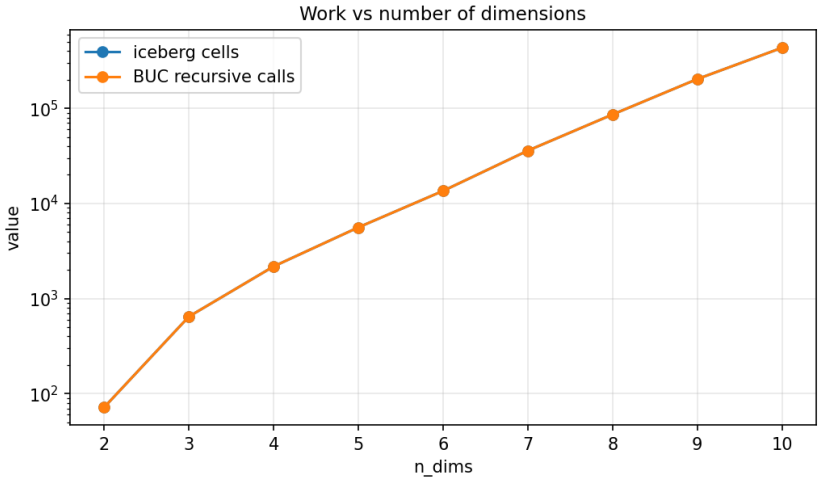



**Figure 4:** Cube size vs. number of dimensions.

| budget | runtime (s) | pages read | I/O read (MB) | disk children |
|---|---|---|---|---|
| 64 KB | 34.80 | 14,474 | 1,221.9 | 4,155 |
| 256 KB | 17.16 | 6,996 | 1,041.5 | 1,337 |
| 1 MB | 10.70 | 3,701 | 767.1 | 325 |
| 8 MB | 6.16 | 594 | 216.7 | 8 |

→ Shrinking the budget from 8 MB to 64 KB forces 4,155 disk-resident recursions and 1.2 GB of page reads, but the algorithm **still completes with identical output** — the classic time/space trade-off of external aggregation.

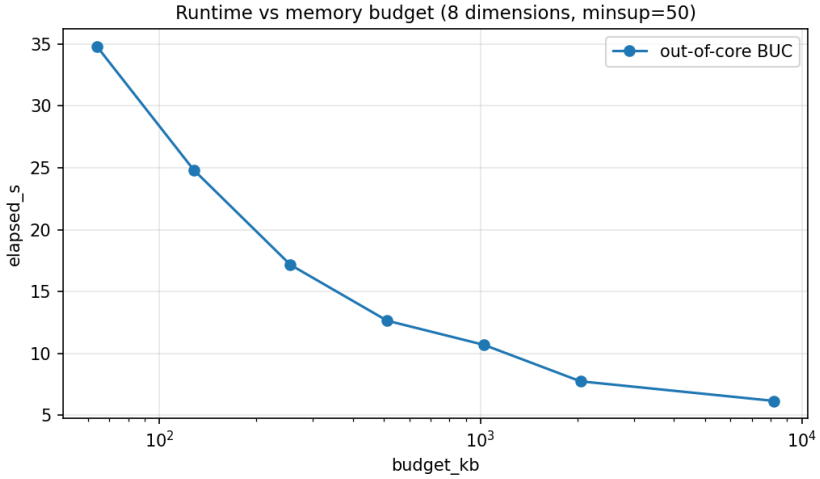



**Figure 5:** Paged BUC runtime vs. memory budget.

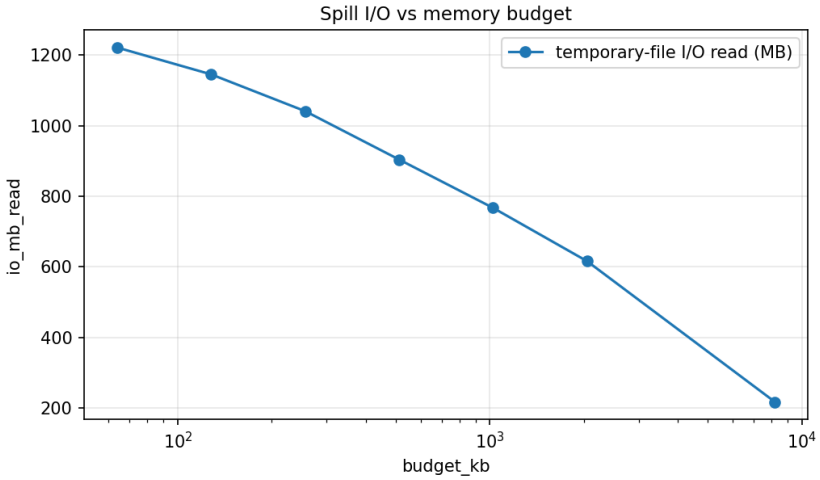



**Figure 6:** Paged BUC I/O volume vs. memory budget.

**Contribution of each optimization** (`benchmark_optimizations.csv`, median of 3 runs, 8 dims, minsup = 50):

**Table 5:** Ablation of BUC optimizations (8 dims, minsup = 50).

| configuration | runtime (s) | pruned partitions | skipped sorts |
|---|---|---|---|
| none | 6.26 | 136,948 | 0 |
| +reorder | 5.60 | 97,600 | 0 |
| +precount prune | 6.14 | 97,600 | 10,759 |
| +constant shortcut | 6.66 | 97,600 | 10,759 |
| +collapse (all) | 8.33 | 97,600 | 10,759 |

→ **Reordering is the single most effective optimization** (−10% runtime, −29% partitions). Pre-count pruning and the constant shortcut reduce work (10,759 pre-count skips, 267 sort skips) but their bookkeeping overhead roughly cancels out at this data size; node collapsing adds overhead because output rows are already few. All configurations produce the identical 86,043-cell cube.

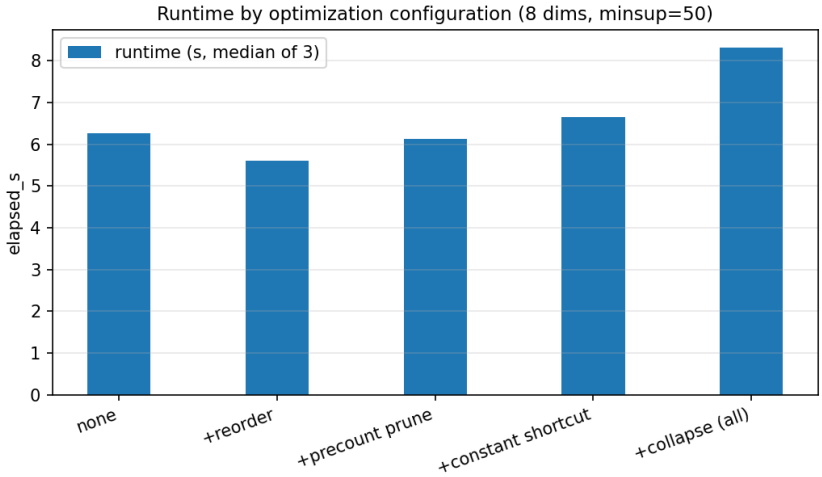



**Figure 7:** Runtime contribution of each BUC optimization.

# 4 AOI — Attribute-Oriented Induction

## 4.1 Concept hierarchies

Ten concept hierarchies were defined as CSV trees (`part2/hierarchies/`), e.g.:

- `education`: 17 values → {High school or less, Some college/Associate, Bachelors, Graduate degree} → ANY
- `class_of_worker`: 8 → {Private sector, Government, Self-employed, Unpaid/Not in universe} → ANY
- `marital_status`: 7 → {Ever married, Never married} → ANY
- `age_group`: 7 → {Young adult, Prime working age, Older working age, Senior} → ANY
- plus `race`, `employment_status`, `tax_filer_status`, `household_summary`, `citizenship`, `weeks_worked_band`.

Each hierarchy is validated to be a strict tree (every value has exactly one parent, single ANY root).

### 4.2 Algorithm

`part2/src/aoi.py` implements classic AOI:

1. **Attribute removal** — attributes with too many distinct values to generalize usefully are dropped.
2. **Concept-tree climbing** — iteratively generalize the attribute with the most distinct values (one level up its hierarchy) until the number of distinct tuples $\leq$ `max_relations`.
3. **Threshold control** — `attr_threshold` (max distinct values per attribute) and `max_relations` (max output rules) bound the output size.
4. **Class comparison** — `aoi_class_rules()` runs AOI on a target class (income >50K) vs. a contrast class ($\leq$50K) and computes:
   - **t-weight** = count of the rule within the target class / total target tuples (how much of the target this rule covers), and
   - **d-weight** = count within the contrast class / total contrast tuples (how specific the rule is to the target).

### 4.3 Results — discriminant rules for income >50K

(`part2/outputs/aoi_rules.csv`, 32 rules from 18,568 target tuples, `attr_threshold`= 12, `max_relations`= 40; top rows shown)

**Table 6:** Top AOI discriminant rules for the class income >50K (t-weight = coverage of target class; d-weight = prevalence in contrast class).

| education | class_of_worker | marital_status | age_group | t-w (%) | d-w (%) |
|---|---|---|---|---|---|
| ANY | Private sector | Ever married | Prime working age | 27.99 | 14.14 |
| ANY | Private sector | Ever married | Older working age | 23.68 | 7.51 |
| ANY | Self-employed | Ever married | Older working age | 8.75 | **1.89** |
| ANY | Government | Ever married | Older working age | 8.57 | 2.23 |
| ANY | Self-employed | Ever married | Prime working age | 6.46 | 2.06 |
| ANY | Private sector | Never married | Prime working age | 4.39 | 4.43 |
| ANY | Unpaid/Not in univ. | Ever married | Senior | 3.84 | 10.64 |

**Interpretation.** High-income (>50K) individuals are characterized by:

- **Ever married** (appears in every top rule) and **prime/older working age** — the top two rules alone cover 51.7% of all >50K tuples (t-weights 28.0% + 23.7%).
- **Self-employed / Government workers in the older bracket** are the most *discriminative* patterns: t-weight 8.75% vs. d-weight only 1.89% — i.e. this profile is 4.6× more prevalent among high earners than low earners.
- By contrast, "Unpaid/Not in universe, Ever married, Senior" has a high d-weight (10.6%) relative to its t-weight (3.8%) — it is a predominantly *low-income* profile that still appears among >50K (likely retirement/pension income).
- "Never married, Young adult" shows d-weight 10.45% vs. t-weight 0.56% — overwhelmingly a $\leq$50K profile.

### 4.4 Threshold sensitivity

(`part2/outputs/aoi_thresholds.csv`) — tightening `max_relations` from 50 → 5 forces more climbing (7 → 5 climbs) and shrinks the output from 40 → 2 rules, at negligible cost (<1 s). AOI gives direct control over output size, unlike BUC where output size is controlled only indirectly via minsup.

### 4.5 BUC vs AOI comparison

(`part2/outputs/buc_vs_aoi.csv`, same 5 attributes, 299,285 tuples)

**Table 7:** BUC vs. AOI on the same data subset (5 attributes).

| aspect | BUC (in-mem) | BUC (paged) | AOI |
|---|---|---|---|
| input tuples | 299,285 | 299,285 | 299,285 |
| output rows | 5,602 | 5,602 | 40 |
| runtime (s) | 0.97 | 0.86 | **0.12** |
| output bounded by | minsup | minsup | max_relations |
| needs concept hierarchies | no | no | yes |
| exact iceberg counts | yes | yes | yes (per rule) |
| out-of-core (disk spill) | no | yes | no |

→ **BUC answers "which cells are frequent, and how frequent"** (exact counts, exponential output, needs minsup tuning); **AOI answers "what are the general profiles in this data"** (compact, human-readable rules, hierarchy-driven, 8× faster here, but requires domain knowledge to build the hierarchies and loses detail by generalizing).

## 5 Conclusions

1. Both BUC variants were **verified correct** against DuckDB CUBE on all 255 tested dimension combinations — the paged variant is a faithful out-of-core implementation (identical output at every budget).

2. **minsup is the dominant cost lever** for BUC: 29× smaller cube and 4× faster runtime from minsup 5 → 1000.

3. **Dimension reordering** is the most valuable optimization; pre-count pruning and constant shortcuts reduce work measurably (10,759 / 267 skips) but their overhead offsets gains at this scale.

4. The **out-of-core variant trades I/O for memory**: at a 64 KB budget it reads 1.2 GB of pages (4,155 disk children) yet still finishes with identical results — essential when the cube exceeds RAM.

5. **AOI produces compact, interpretable rules** (40 rows vs. 5,602 cells) with t/d-weights that clearly separate high- vs. low-income profiles (e.g. self-employed older workers: t=8.75% vs. d=1.89%).

6. BUC and AOI are **complementary**: BUC for exhaustive, exact iceberg analysis; AOI for summarized, human-readable characterization.